spam_filtering

January 27, 2020

1 Bayesian classifier for spam identification

1.0.1 1. Download data: 1899 spam and 2501 nonspam emails

```
In [1]: data_root = "/Users/liza/Documents/spam_repo/ML_for_Hackers/03-Classification/data"
```

```
In [2]: import os
        non_spam_filenames = [os.path.join(data_root, "easy_ham", name) for name in os.listdir(o
        spam_filenames = [os.path.join(data_root, "spam", name) for name in os.listdir(os.path.j
                          [os.path.join(data_root, "spam_2", name) for name in os.listdir(os.path
```

```
In [3]: len(spam_filenames)
```

```
Out[3]: 1899
```

```
In [4]: len(non_spam_filenames)
```

```
Out[4]: 2501
```

Non-spam example: Theo Van Dinter wrote: > On Thu, Aug 22, 2002 at 07:27:52AM -0700, Marc Perkel wrote: > »Has anyone though of the idea of live updates of rules after release? The »idea being that the user can run a cron job once a week or so and get the »new default rule set. This would allow us to react faster to: > > > I suggested this a few months ago. I don't remember the details of what > came out of it except that it would only be useful for non-eval rules > since those require code changes.

Spam example: Protect your financial well-being. Purchase an Extended Auto Warranty for your Car today. CLICK HERE for a FREE no obligation quote. http://www.newnamedns.com/warranty/

Car troubles always seem to happen at the worst possible time. Protect yourself and your family with a quality Extended Warranty for your car, truck, or SUV, so that a large expense cannot hit you all at once. We cover most vehicles with less than 150,000 miles.

Buy DIRECT! Our prices are 40-60

We offer fair prices and prompt, toll-free claims service. Get an Extended Warranty on your car today.

Warranty plan also includes:

1) 24-Hour Roadside Assistance. 2) Rental Benefit. 3) Trip Interruption Intervention. 4) Extended Towing Benefit.

CLICK HERE for a FREE no obligation quote. http://www.newnamedns.com/warranty/

or html pages (another spam example):

```
<B>Just Fill Out Our Simple Form and They Will Have to Compete For Your Business...</FONT><FONT
</FONT><FONT  COLOR="#0000ff" BACK="#000000" style="BACKGROUND-COLOR: #000000" SIZE=3 PTSIZE=12
CLICK HERE FOR FORM</A></B></FONT><FONT  COLOR="#000000" BACK="#000000" style="BACKGROUND-COLOR:
<BR>
</FONT><FONT  COLOR="#ffff80" BACK="#000000" style="BACKGROUND-COLOR: #000000" SIZE=2 PTSIZE=10
</FONT><FONT  COLOR="#ff0000" BACK="#000000" style="BACKGROUND-COLOR: #000000" SIZE=2 PTSIZE=10
Refinance<BR>
Debt Consolidation<BR>
Home Improvement <BR>
Second Mortgages<BR>
No Income Verification</B></I><BR>
</FONT><FONT  COLOR="#000000" BACK="#000000" style="BACKGROUND-COLOR: #000000" SIZE=2 PTSIZE=10
</FONT><FONT  COLOR="#0000ff" BACK="#000000" style="BACKGROUND-COLOR: #000000" SIZE=3 PTSIZE=12
<BR>
```

1.0.2 2. Looking at the word list

```
In [5]: import re

        def get_words(filename):
            f = open(filename, errors="replace")
            for line in f:
                if line == "\n":
                    break
            words = []

            for line in f:
                line_words = re.split(r"[^a-zA-Z']+",line)
                words.extend(w.lower() for w in line_words if w != "")
            return words
```

```
In [6]: data = []
        for filename in non_spam_filenames:
            data.append([filename, get_words(filename), False])
        for filename in spam_filenames:
            data.append([filename, get_words(filename), True])
```

```
In [7]: import pandas as pd
        data = pd.DataFrame(data, columns = ['Filename', 'Words', 'IsSpam'])
```

```
In [88]: data.head()
```

```
Out[88]:                                              Filename  \
         0  /Users/liza/Documents/spam_repo/ML_for_Hackers...
         1  /Users/liza/Documents/spam_repo/ML_for_Hackers...
         2  /Users/liza/Documents/spam_repo/ML_for_Hackers...
         3  /Users/liza/Documents/spam_repo/ML_for_Hackers...
```

```
4  /Users/liza/Documents/spam_repo/ML_for_Hackers...

                                                Words  IsSpam
0  [in, a, message, dated, am, jamesr, best, com,...   False
1  [hiya, i, always, seem, to, get, errors, when,...   False
2  [i, don't, know, how, one, can, expect, better...   False
3  [tim, peters, wrote, i've, run, no, experiment...   False
4  [at, am, on, jim, whitehead, wrote, great, thi...   False
```

```
In [91]: data.iloc[0]["Words"][1:10]
```

```
Out[91]: ['a', 'message', 'dated', 'am', 'jamesr', 'best', 'com', 'writes', 'this']
```

1.0.3 3. Split into train and test

```
In [10]: from sklearn.model_selection import train_test_split
         train_data, test_data = train_test_split(data, test_size=0.2, random_state=42)
```

```
In [11]: spam_size = len(train_data[train_data.IsSpam])
         spam_size
```

```
Out[11]: 1541
```

1.0.4 4. Choose words frequent for spam emails, but not for nonspam (spam indicating words)

```
In [13]: from collections import Counter
         words_in_spam = Counter()
         num_words_in_spam = 0
         for words in train_data[train_data.IsSpam].Words:
             for word in words:
                 num_words_in_spam += 1
                 words_in_spam[word] += 1
```

```
In [93]: frequent_for_spam = [[x[0], x[1]/num_words_in_spam] for x in words_in_spam.items()]
         frequent_for_spam[0:10]
```

```
Out[93]: [['have', 0.002347646152349832],
          ['you', 0.009442338116368652],
          ['checked', 1.784603688597364e-05],
          ['your', 0.006205959327097333],
          ['personal', 0.0003024903252172532],
          ['credit', 0.0005915961227700261],
          ['reports', 0.00018649108545842454],
          ['recently', 6.335343094520641e-05],
          ['if', 0.001997863829384749],
          ['are', 0.0027893355652776797]]
```

```
In [15]: from collections import Counter
         words_in_nonspam = Counter()
```

```
        num_words_in_nonspam = 0
        for words in train_data[~train_data.IsSpam].Words:
            for word in words:
                num_words_in_nonspam += 1
                words_in_nonspam[word] += 1
```

```
In [94]: frequent_for_nonspam = [[x[0], x[1]/num_words_in_nonspam] for x in words_in_nonspam.ite
         frequent_for_nonspam[0:10]
```

```
Out[94]: [['if', 0.0035828948385238472],
          ['the', 0.04170442756815111],
          ['frequency', 1.1708806661842638e-05],
          ['of', 0.019031494348159025],
          ['my', 0.0028428982574953926],
          ["laptop's", 7.025283997105583e-06],
          ['disk', 0.00011240454395368933],
          ['chirps', 2.3417613323685276e-06],
          ['are', 0.004613269824765999],
          ['any', 0.0014682843553950 67]]
```

```
In [18]: frequent_for_nonspam = dict(frequent_for_nonspam)
         relative_frequency = []
         for word, freq in frequent_for_spam:
             relative_frequency.append([word, freq - frequent_for_nonspam.get(word, 0)])
```

```
In [19]: relative_frequency.sort(key = lambda x: -x[1])
```

```
In [97]: relative_frequency[0:5]
```

```
Out[97]: [['d', 0.03717647680606948],
          ['font', 0.030915638791512115],
          ['td', 0.015154809361952716],
          ['br', 0.01402392778102576],
          ['b', 0.013431218910136657]]
```

```
In [113]: relative_frequency[72]
```

```
Out[113]: ['money', 0.0009101027195685574]
```

```
In [114]: relative_frequency[79]
```

```
Out[114]: ['business', 0.0008289414843792943]
```

1.0.5 Choose 12 words indicating spam:

chosen 12 clearly spam related words from the first 200 of more frequent words:

```
In [75]: spam_words = ['business', 'money', 'content', 'click', 'family', 'receive', 'credit', '
```

Some more straightforward ways to define spam indicating words:

```
In [37]: top_spam_words = [x[0] for x in relative_frequency[0:12]]

In [87]: top200_spam_words = [x[0] for x in relative_frequency[0:200]]

In [38]: top_spam_words

Out[38]: ['d',
          'font',
          'td',
          'br',
          'b',
          'size',
          'p',
          'tr',
          'nbsp',
          'color',
          'face',
          'width']
```

1.0.6 4. Find conditional distribution: $\mathbb{P}(spam\_words\_appearance|Spam)$ and $\mathbb{P}(spam\_words\_appearance|NonSpam)$

```
In [76]: top_spam_word_ps = {}

         for word in spam_words:
         #for word in top_spam_words:
             mails_with_word = train_data.apply(lambda x: word in x['Words'], axis=1)
             p_spam = len(train_data[mails_with_word & train_data.IsSpam]) / spam_size
             p_nonspam = len(train_data[mails_with_word & ~train_data.IsSpam]) / nonspam_size
             top_spam_word_ps[word] = (p_spam,  p_nonspam)

         top_spam_word_ps

Out[76]: {'business': (0.26411421155094095, 0.05305709954522486),
          'money': (0.2290720311486048, 0.04295098534613441),
          'content': (0.34523036988968203, 0.07225871652349672),
          'click': (0.5373134328358209, 0.16422435573521982),
          'family': (0.16807268007787152, 0.03335017685699848),
          'receive': (0.30499675535366644, 0.019201616978271854),
          'credit': (0.16612589227774172, 0.008590197069226882),
          'grants': (0.010382868267358857, 0.004547751389590703),
          'insurance': (0.0837118754055808, 0.00404244567963618),
          'wish': (0.2109020116807268, 0.017685699848408287),
          'price': (0.13043478260869565, 0.014653865588681153),
          'special': (0.15574302401038287, 0.019706922688226377)}
```

1.0.7 5. Find prior probabilities (fraction of spam emails in train data)

```
In [58]: spam_pr = spam_size/(spam_size + nonspam_size)
         nonspam_pr = nonspam_size/(spam_size + nonspam_size)
```

1.0.8 6. Bayesian prediction

MAP estimator $\hat{\theta}$: $\theta = 1$ if email is Spam and $\theta = 0$ is email is NonSpam.

$$\hat{\theta} = arg\max \mathbb{P}(\theta | observed\_spam\_words\_appearances)$$

That is, if

$$\mathbb{P}(Spam | observed\_spam\_words\_appearances) > \mathbb{P}(NonSpam | observed\_spam\_words\_appearances),$$

then we decide $\hat{\theta} = Spam$.

By Bayes rule, we compute: if

$$\mathbb{P}_{prior}(Spam) \cdot \mathbb{P}(spam\_words\_appearance | Spam) > \mathbb{P}_{prior}(NonSpam)\mathbb{P}(spam\_words\_appearance | NonSpam),$$

then we decide $\hat{\theta} = Spam$.

Note! Denominators are the same, so we do not compute them.

```
In [77]: class_label = []
         for email in test_data.Words:
             count_spam = 1
             count_nonspam = 1
             for spam_word in spam_words:
         #    for spam_word in top_spam_words:
                 if spam_word in email:
                     count_spam *= top_spam_word_ps[spam_word][0]
                     count_nonspam *= top_spam_word_ps[spam_word][1]
                 else:
                     count_spam *= (1 - top_spam_word_ps[spam_word][0])
                     count_nonspam *= (1 - top_spam_word_ps[spam_word][1])
             if spam_pr*count_spam > nonspam_pr*count_nonspam:
                 class_label.append(True)
             else:
                 class_label.append(False)
```

```
In [78]: i = 0
         false_positive_rate = 0
         false_negative_rate = 0
         true_spam_rate = 0
         true_nonspam_rate = 0

         for spam_label in test_data.IsSpam:
             if spam_label is True and class_label[i] is False:
                 false_negative_rate += 1
```

```
            if spam_label is False and class_label[i] is True:
                false_positive_rate += 1
            if spam_label is False and class_label[i] is False:
                true_nonspam_rate += 1
            if spam_label is True and class_label[i] is True:
                true_spam_rate += 1
            i += 1
```

Non-detected spam

```
In [79]: false_negative_rate/len(test_data)
```

```
Out[79]: 0.14545454545454545
```

Falsely detected spam (good emails)

```
In [80]: false_positive_rate/len(test_data)
```

```
Out[80]: 0.04318181818181818
```

Correctly detected spam

```
In [81]: true_spam_rate/len(test_data)
```

```
Out[81]: 0.26136363636363635
```

Correctly detected good emails

```
In [82]: true_nonspam_rate/len(test_data)
```

```
Out[82]: 0.55
```

```
In [84]: 0.145 + 0.043 + 0.26 + 0.55
```

```
Out[84]: 0.998
```

```
In [85]: test_spam_size = len(test_data[test_data.IsSpam])
         test_spam_size
```

```
Out[85]: 358
```

```
In [86]: test_nonspam_size = len(test_data[~test_data.IsSpam])
         test_nonspam_size
```

```
Out[86]: 522
```

```
In [ ]:
```